TIPE sur le looping d'un avion

Pierre CATHELAIN

juin 2026

Le plan

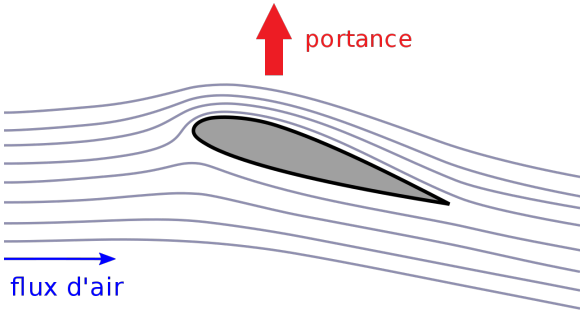
1. Définitions
2. Étude théorique
3. Modélisation informatique
4. Meilleure configuration possible
5. Conclusion

Définitions

Portance

La portance

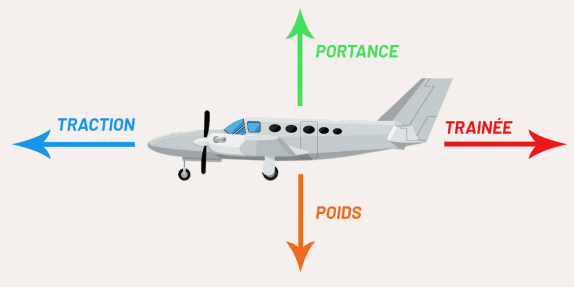
La portance est la force qui permet à un objet de se maintenir ou de s'élever dans l'air. Elle est surtout utilisée pour parler des ailes d'un avion : l'air qui passe autour des ailes crée une force vers le haut qui compense le poids de l'avion.



Traînée

La traînée

En mécanique des fluides, la traînée est la force qui s'oppose au mouvement d'un corps dans un liquide ou un gaz et agit comme un frottement.



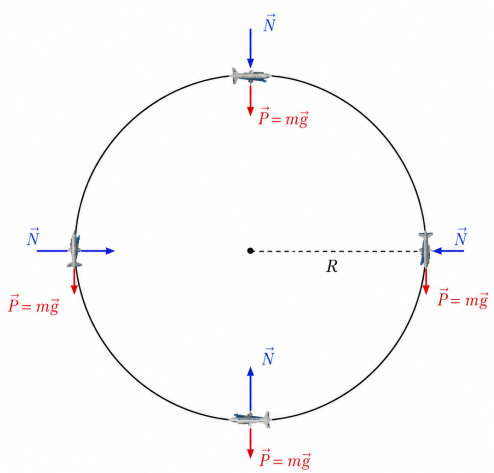
Décrochage

Le décrochage

En aéronautique, le décrochage est un phénomène qui se produit lorsqu'un avion perd brutalement sa portance. Il se produit à cause d'un angle trop élevé avec l'horizontal.

Étude théorique

Version très simple :



Version très simple

- vitesse v constante
- l'avion est modélisé comme une masse ponctuelle m
- Forces :
  1. poids : $\vec{P} = m\vec{g}$
  2. portance remplacée par une force normale $\vec{N}$
- système = avion
- référentiel terrestre supposé galiléen
- accélération centripète v^2/R

Vitesse nécessaire

La trajectoire la plus "économe" consiste à arriver au sommet avec une portance minimale : $N = 0$.
D'après le PFD

$$v_{haut} = \sqrt{gR}$$

.
L'énergie mécanique se conserve donc d'après le théorème de l'énergie mécanique

$$v_{bas} = \sqrt{5gR}$$

.

Remarque

A.N : (pour un rayon de $400m$)
$v_{bas} \approx 62.6 m.s^{-1}$
$v_{haut} \approx 140 m.s^{-1}$

Facteur de charge (g ressentis)

Le pilote ressent une accélération $n = L/mg$ (en "g").

$$L - mg = mv_{bas}^2/R \implies n = 1 + v_{bas}^2/gR$$
$$\implies n = 1 + 5 = 6$$

Ajout des frottements de l'air

- vitesse v constante
- l'avion est modélisé comme une masse ponctuelle m
- Forces :
  1. poids : $\vec{P} = m\vec{g}$
  2. portance remplacée par une force normale $\vec{N}$
  3. frottement de l'air $\vec{f} = -h\vec{v}$ (modèle linéaire simple)
- système = avion
- référentiel terrestre supposé galiléen
- accélération centripète v^2/R

Vitesse nécessaire

D'après le PFD (en prenant la portance nulle au sommet) on a

$$v_{haut} = \frac{h + \sqrt{h^2 + 4\frac{m^2}{R}g}}{2\frac{m}{R}}$$

.

D'après le théorème de l'énergie mécanique

$$v_{bas} = \sqrt{v_{haut}^2 + 4gR}$$

.

Remarque

A.N : (pour un rayon de $400m$, $m \approx 1000kg$ et $h \approx 10^3 kg/s$)

$v_{bas} \approx 410 m.s^{-1}$

$v_{haut} \approx 428 m.s^{-1}$

Portance réelle

- vitesse v constante
- l'avion est modélisé comme une masse ponctuelle m
- Forces :
  1. poids : $\vec{P} = m\vec{g}$
  2. portance : $L = \frac{1}{2}\rho S C_L v^2$
- système = avion
- référentiel terrestre supposé galiléen
- accélération centripète v^2/R

Vitesse nécessaire

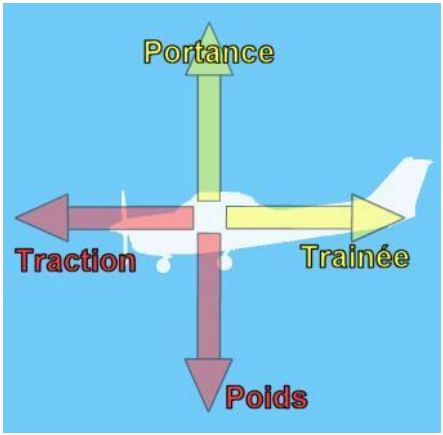
D'après le PFD

$$v_{min} = \sqrt{\frac{mg}{\frac{m}{R} - \frac{1}{2}\rho SC}}$$

.

Remarque | | On remarque la condition $R < \frac{2m}{\rho SC}$

Forces réelles



- vitesse v constante
- l'avion est modélisé comme une masse ponctuelle m
- Forces :
  1. poids : $\vec{P} = m\vec{g}$
  2. portance : $L = \frac{1}{2}\rho S C_L v^2$
  3. la traction du moteur $\vec{T}$
  4. la traînée $D == \frac{1}{2}\rho S C_L v^2$
- système = avion
- référentiel terrestre supposé galiléen
- accélération centripète v^2/R

Vitesse

On obtient l'équation différentielle suivante :

$$\frac{dv}{d\theta} = \frac{R}{mv}\left(T - \frac{1}{2}\rho S C_D v^2 - mg\sin\theta\right)$$

.

1er résolution

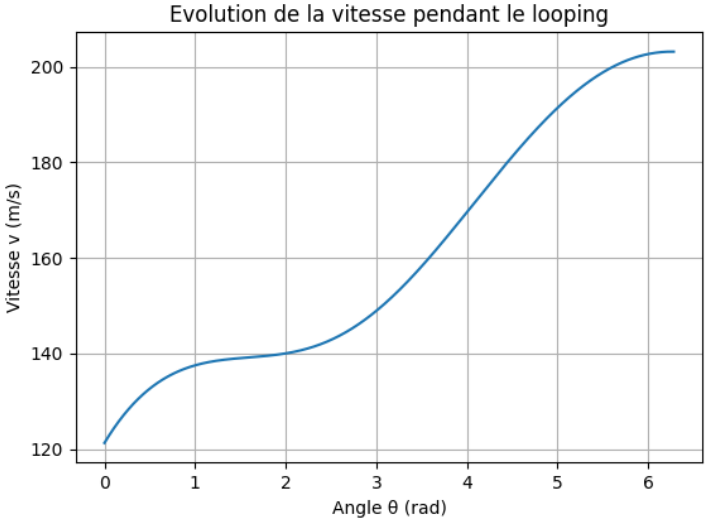
On pose $k = \frac{1}{2}\rho S C_D \implies \frac{dv}{d\theta} = \frac{R}{mv}(T - kv^2 - mg\sin\theta)$.

A l'aide de changement de variable, facteur intégrant et résolution on obtient :

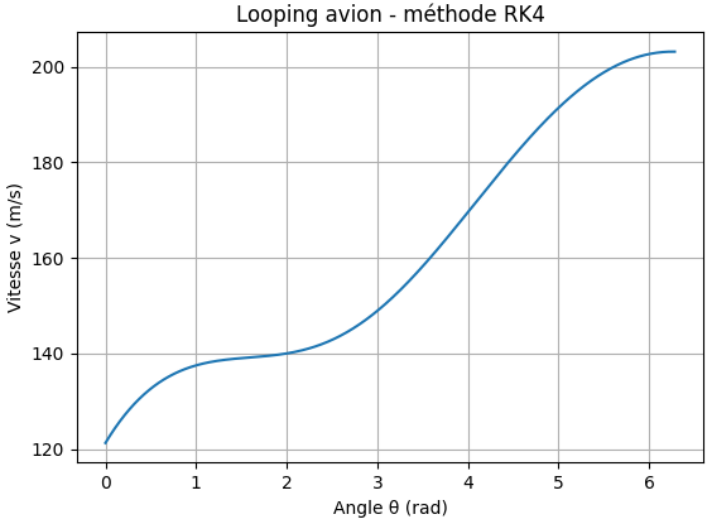
$$v(\theta) = \sqrt{Ae^{-\frac{2Rk}{m}\theta} + \frac{T}{k} - \frac{2Rg}{(\frac{2Rk}{m})^2 + 1}\left(\frac{2Rk}{m}\sin\theta - \cos\theta\right)}$$

.

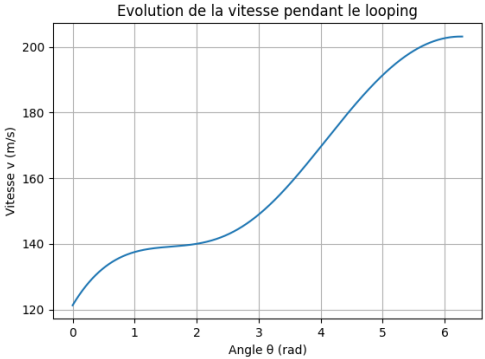
Résolution numérique (Euler)

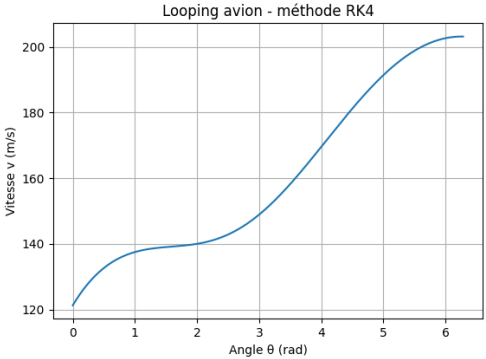


Résolution numérique (Runge-Kutta d'ordre 4)

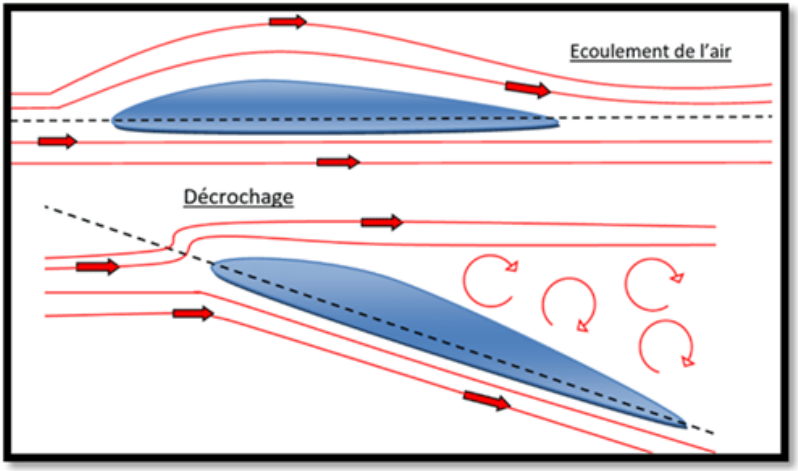


Comparatif


Evolution de la vitesse pendant le looping
Vitesse v (m/s)
Angle θ (rad)
120
140
160
180
200
0
1
2
3
4
5
6

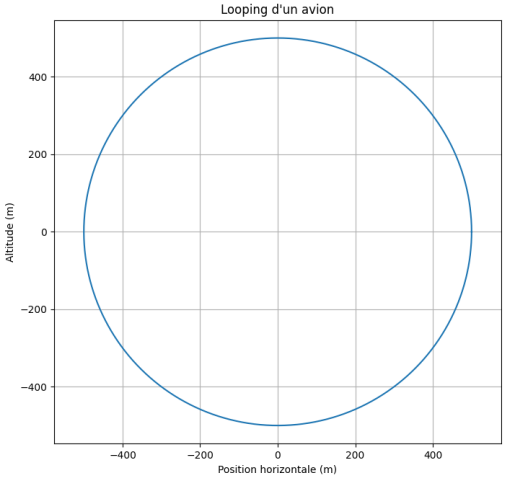

Looping avion - méthode RK4
Vitesse v (m/s)
Angle θ (rad)
120
140
160
180
200
0
1
2
3
4
5
6

Le décrochage

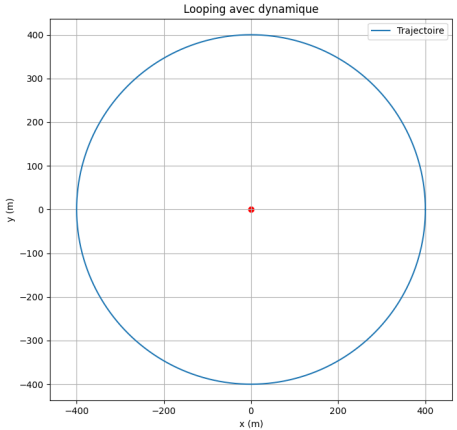


Modélisation informatique

Modélise le looping comme un cercle

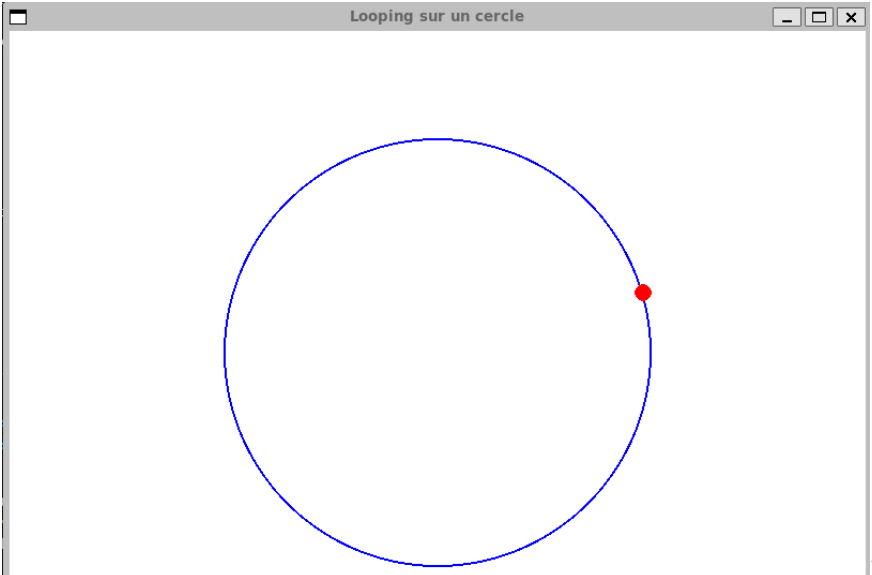


Avec vitesse et accélération centripète

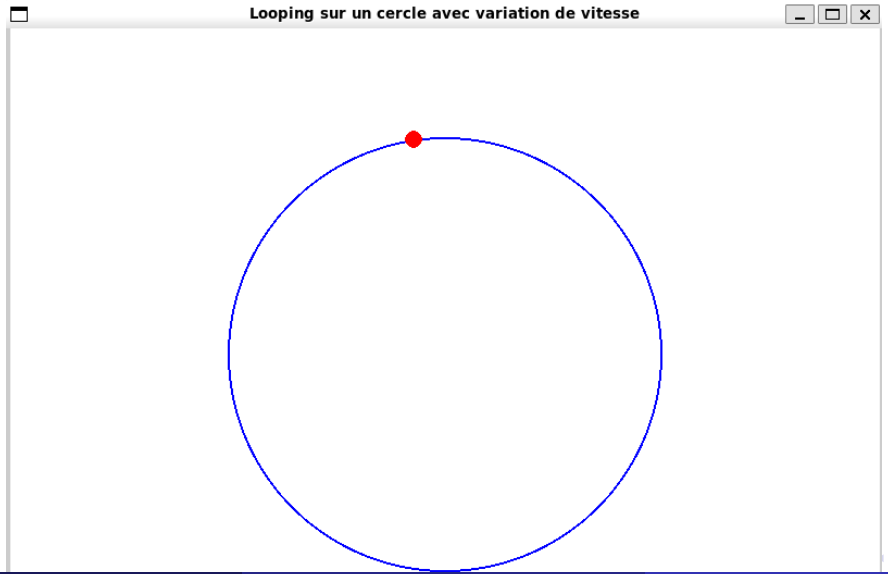


Accélération centripète : $56.25 m/s^2$
Facteur de charge : $5.73 g$

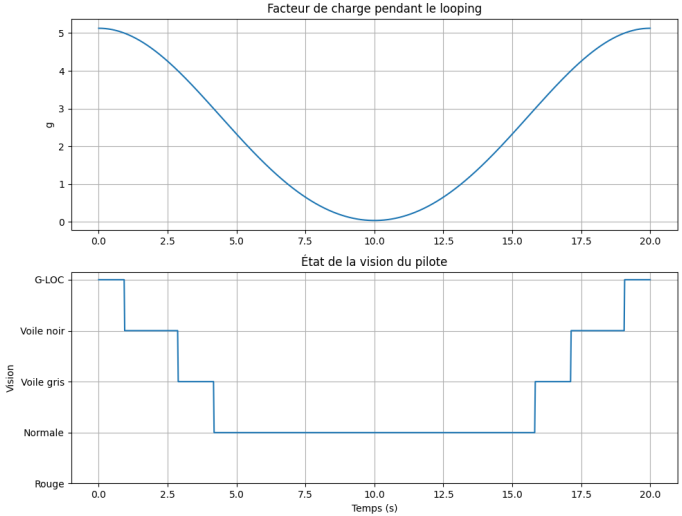
Modélisation 2D



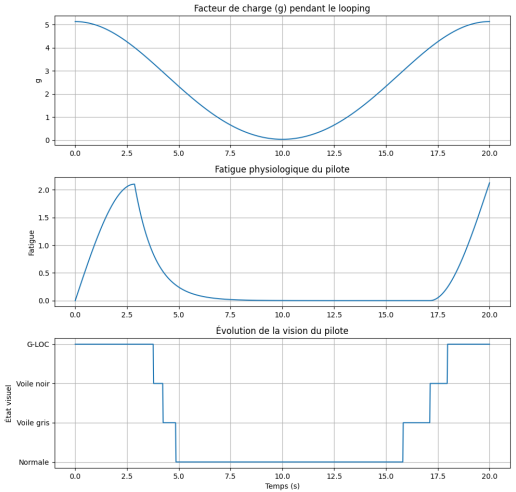
Avec variation de vitesse



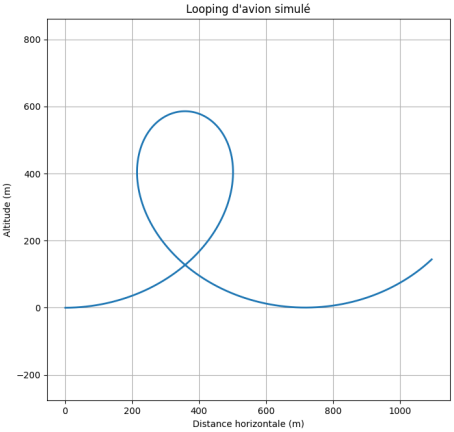
Évolution des g et de la vision du pilote



Avec la fatigue physiologique

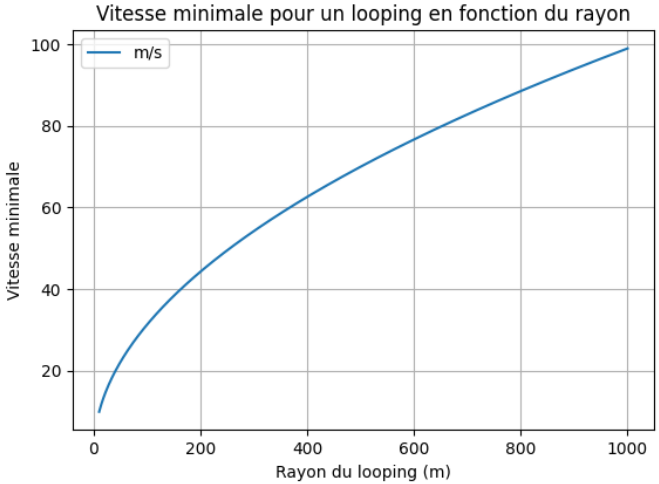


Trajectoire réelle

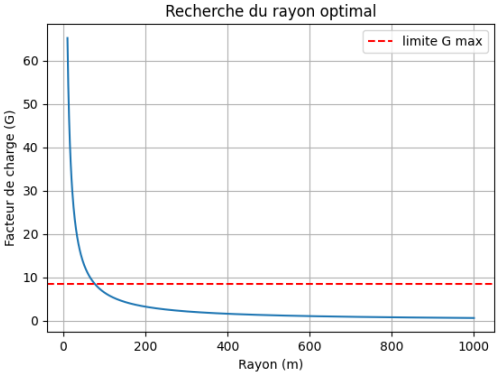


Meilleure configuration possible

Vitesse minimale version simple



Rayon optimal minimal



- nbr de g max : 8.5
- Rayon optimal minimal : $77.4m$

Surface alaire

La surface des ailes S intervient dans la formule de protance : $L = \frac{1}{2}\rho S C_L v^2$.

Pour maintenir l'accélération centripète nécessaire on doit avoir $mv^2/R = mg + L$

$$\Longrightarrow S = \frac{2\left(\frac{mv^2}{R} - mg\right)}{\rho v^2 C_L}$$

Remarque $\Big|\Big|$ $\underline{\text{A.N :}}$ ()

$$S \approx 4m^2$$

Conclusion

Annexes

```python
import numpy as np
import matplotlib.pyplot as plt

rayon = 500    #en metres
nb_points = 1000

theta = np.linspace(0, 2*np.pi, nb_points)

x = rayon * np.cos(theta)
y = rayon * np.sin(theta)

plt.figure(figsize=(8,8))
plt.plot(x, y)
plt.title("Looping d'un avion")
plt.xlabel("Position horizontale (m)")
plt.ylabel("Altitude (m)")
plt.axis('equal')
plt.grid()
plt.show()

"""
Ce que fait ce programme
Modélise le looping comme un cercle.
"""
```

```python
import numpy as np
import matplotlib.pyplot as plt

R = 400    # m
v = 150     # m/s
g = 9.81

T = 2*np.pi*R / v
N = 1000
t = np.linspace(0, T, N)

omega = v / R
theta = omega * t

x = R * np.cos(theta)
y = R * np.sin(theta)

ac = v**2 / R


plt.figure(figsize=(8,8))
plt.plot(x, y, label="Trajectoire")
plt.scatter([0], [0], color='red')
plt.axis('equal')
plt.grid()
plt.legend()
plt.title("Looping avec dynamique")
plt.xlabel("x (m)")
plt.ylabel("y (m)")
plt.show()


"""
```

```python
import pygame
import math

pygame.init()

largeur = 800
hauteur = 600

ecran = pygame.display.set_mode((largeur, hauteur))

pygame.display.set_caption("Looping sur un cercle")

BLANC = (255, 255, 255)
ROUGE = (255, 0, 0)
BLEU = (0, 0, 255)

clock = pygame.time.Clock()

centre_x = largeur // 2
centre_y = hauteur // 2

rayon = 200

angle = 0

running = True

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    ecran.fill(BLANC)
    pygame.draw.circle(ecran,BLEU,(centre_x, centre_y),rayon,2)
    rad = math.radians(angle)
    x = centre_x + rayon * math.cos(rad)
    y = centre_y + rayon * math.sin(rad)
    pygame.draw.circle(ecran,ROUGE,(int(x), int(y)),8)
    angle += 2
    pygame.display.flip()
    clock.tick(60)

pygame.quit()


"""
```

```python
import pygame
import math

pygame.init()

largeur = 800
hauteur = 600

ecran = pygame.display.set_mode((largeur, hauteur))
pygame.display.set_caption("Looping sur un cercle avec variation de vitesse")

BLANC = (255, 255, 255)
ROUGE = (255, 0, 0)
BLEU = (0, 0, 255)

clock = pygame.time.Clock()

centre_x = largeur // 2
centre_y = hauteur // 2

rayon = 200

angle = 0

vitesse_base = 2

running = True

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    ecran.fill(BLANC)
    pygame.draw.circle(ecran,BLEU,(centre_x, centre_y),rayon,2)
    rad = math.radians(angle)
    x = centre_x + rayon * math.cos(rad)
    y = centre_y + rayon * math.sin(rad)
    facteur_vitesse = 1 - 0.3 * (-math.sin(rad))
    vitesse = vitesse_base * facteur_vitesse
    pygame.draw.circle(ecran,ROUGE,(int(x), int(y)),8)
    angle += vitesse
    pygame.display.flip()
    clock.tick(60)

pygame.quit()


"""
```

```python
import numpy as np
import matplotlib.pyplot as plt


g0 = 9.81
R = 800
v_min = 90
v_max = 180

T = 20
N = 1000

t = np.linspace(0, T, N)

theta = 2 * np.pi * t / T

v = (v_max + v_min)/2 + ((v_max - v_min)/2) * np.cos(theta)

n = v**2 / (R * g0) + np.cos(theta)

vision = []

for g in n:
    if g < -2:
        vision.append("Rouge")
    elif g < 3:
        vision.append("Normale")
    elif g < 4:
        vision.append("Voile gris")
    elif g < 5:
        vision.append("Voile noir")
    else:
        vision.append("G-LOC")

vision_num = []

mapping = {"Normale": 0,"Voile gris": 1,"Voile noir": 2,"G-LOC": 3,"Rouge": -1}

for v_state in vision:
    vision_num.append(mapping[v_state])

fig, ax = plt.subplots(2, 1, figsize=(10, 8))

ax[0].plot(t, n)
ax[0].set_title("facteur de charge pendant le looping")
ax[0].set_ylabel("g")
ax[0].grid()

ax[1].plot(t, vision_num)
ax[1].set_title("état de la vision du pilote")
ax[1].set_ylabel("Vision")

ax[1].set_yticks([-1, 0, 1, 2, 3])
ax[1].set_yticklabels(["Rouge","Normale", "Voile gris","Voile noir","G-LOC"])

ax[1].grid()

plt.xlabel("Temps (s)")
plt.tight_layout()
plt.show()
```

```
"""

Calcule l'évolution des g subis par le pilote,
```

```python
import numpy as np
import matplotlib.pyplot as plt

g0 = 9.81
R = 800

v_min = 90
v_max = 180

T = 20
N = 1000

t = np.linspace(0, T, N)

theta = 2 * np.pi * t / T

v = (v_max + v_min)/2 + ((v_max - v_min)/2) * np.cos(theta)

n = v**2 / (R * g0) + np.cos(theta)

fatigue = np.zeros(N)

for i in range(1, N):
    g = n[i]
    if g > 4:
        fatigue[i] = fatigue[i-1] + (g - 4) * 0.02
    else:
        fatigue[i] = fatigue[i-1] * 0.98


vision = []

for i in range(N):
    g = n[i]
    f = fatigue[i]
    stress = g + 2 * f
    if stress < 3:
        vision.append(0)
    elif stress < 4:
        vision.append(1)
    elif stress < 5:
        vision.append(2)
    else:
        vision.append(3)


fig, ax = plt.subplots(3, 1, figsize=(10, 10))

ax[0].plot(t, n)
ax[0].set_title("facteur de charge (g) pendant le looping")
ax[0].set_ylabel("g")
ax[0].grid()

ax[1].plot(t, fatigue)
ax[1].set_title("Fatigue physiologique du pilote")
ax[1].set_ylabel("Fatigue")
ax[1].grid()


ax[2].plot(t, vision)
ax[2].set_title("Evolution de la vision du pilote")
ax[2].set_ylabel("Etat visuel")
ax[2].set_yticks([0, 1, 2, 3])
ax[2].set_yticklabels(["Normale","Voile gris","Voile noir","G-LOC"])
ax[2].set_xlabel("Temps (s)")
ax[2].grid()

plt.tight_layout()
plt.show()
```

```python
import numpy as np
import matplotlib.pyplot as plt

g = 9.81  # m/s²
R = np.linspace(10, 1000, 500)
v_min = np.sqrt(g * R)

plt.figure()

plt.plot(R, v_min, label="m/s")

plt.title("Vitesse minimale pour un looping en fonction du rayon")
plt.xlabel("Rayon du looping (m)")
plt.ylabel("Vitesse minimale")
plt.legend()

plt.grid(True)
plt.show()


"""
Ce que fait ce programme
Il teste des rayons de 10 m à 1000 m
Il calcule la vitesse minimale pour chaque rayon
"""
```

```python
import numpy as np
import matplotlib.pyplot as plt

g = 9.81
masse = 1200
v = 80
G_max = 8.5

R = np.linspace(10, 1000, 1000)

n = v**2 / (g * R)

valid = np.where(n <= G_max)[0]

if len(valid) > 0:
    R_opt = R[valid[0]]
    print(R_opt)
else:
    print("Aucun rayon compatible avec ces paramètres")

plt.plot(R, n)
plt.axhline(G_max, color="red", linestyle="--", label="limite G max")
plt.xlabel("Rayon (m)")
plt.ylabel("Facteur de charge (G)")
plt.title("Recherche du rayon optimal")
plt.legend()
plt.grid()
plt.show()
```

```python
import numpy as np
import matplotlib.pyplot as plt

m = 1000
R = 300
g = 9.81

rho = 1.225
S = 16
CD = 0.05

T = 20000

k = 0.5 * rho * S * CD

theta0 = 0
v0 = np.sqrt(5 * g * R)

dtheta = 0.001

N = int(2*np.pi / dtheta)

theta_vals = np.zeros(N)
v_vals = np.zeros(N)

theta = theta0
v = v0

for i in range(N):
    theta_vals[i] = theta
    v_vals[i] = v
    f = (R / (m * v)) * (T - k * v**2 - m * g * np.sin(theta))
    v = v + dtheta * f
    theta = theta + dtheta


plt.plot(theta_vals, v_vals)
plt.xlabel("Angle θ (rad)")
plt.ylabel("Vitesse v (m/s)")
plt.title("Evolution de la vitesse pendant le looping")
plt.grid()
plt.show()

"""
```

```python
import numpy as np
import matplotlib.pyplot as plt

m = 1000
R = 300
g = 9.81
rho = 1.225
S = 16
CD = 0.05
T = 20000
k = 0.5 * rho * S * CD

theta0 = 0
v0 = np.sqrt(5 * g * R)
h = 0.001
N = int(2*np.pi / h)-

theta_vals = np.zeros(N)
v_vals = np.zeros(N)

def f(theta, v):
    return (R / (m * v)) * (T- k * v**2- m * g * np.sin(theta))

theta = theta0
v = v0


for i in range(N):
    theta_vals[i] = theta
    v_vals[i] = v
    k1 = f(theta, v)
    k2 = f(theta + h/2,v + h*k1/2)
    k3 = f(theta + h/2,v + h*k2/2)
    k4 = f(theta + h,v + h*k3)
    v = v + (h/6)*(k1 + 2*k2 + 2*k3 + k4)
    theta = theta + h


plt.plot(theta_vals, v_vals)
plt.xlabel("Angle θ (rad)")
plt.ylabel("Vitesse v (m/s)")
plt.title("Looping avion - méthode RK4")
plt.grid()
plt.show()


"""
```

```python
import numpy as np
import matplotlib.pyplot as plt

g = 9.81
n = 3.5
V0 = 120.0
dt = 0.01
tmax = 25

x = 0.0
y = 0.0
V = V0
gamma = 0.0

X = []
Y = []

for k in range(int(tmax/dt)):
    dV = -g * np.sin(gamma) * dt
    V += dV
    if V < 20:
        break
    an = (n - 1.0) * g
    dgamma = an / V * dt
    gamma += dgamma
    x += V * np.cos(gamma) * dt
    y += V * np.sin(gamma) * dt
    X.append(x)
    Y.append(y)


plt.figure(figsize=(8,8))
plt.plot(X, Y, linewidth=2)
plt.axis('equal')
plt.grid(True)
plt.xlabel("Distance horizontale (m)")
plt.ylabel("Altitude (m)")
plt.title("Looping d'avion simulé")
plt.show()
"""
trajectoire réelle à l'aide du facteur de charge
```